Muscle Memory for Machines: Cognitive Motor Learning for Autonomous Desktop Agents

Victor Santiago Montaño Diaz
Independent Researcher

***Abstract*—Current AI computer use agents treat every click as a first-time click. No matter how many times an agent clicks the same button, the movement takes the same amount of time. Humans, by contrast, get faster with practice. We present a cross-platform computer use system, exposed as a Model Context Protocol (MCP) server, whose key innovation is a spatiotemporal motor memory cache that lets any MCP-compatible AI agent develop "muscle memory." The cache combines eight mechanisms: R-Tree spatial indexing, Power Law of Practice speed-up, Fitts's Law movement timing, WindMouse trajectory adaptation, temporal decay, EMA position smoothing, action sequence prediction, and cross-session persistence. A three-layer resolution hierarchy ensures that *every* mouse interaction builds cache entries, regardless of application or platform. The system exposes 20 tools across four platforms (Windows, WSL2, Linux with X11 and Wayland, and macOS) and requires no SDK integration from the host agent. Testing on real desktop applications shows cache hits growing across sessions and across independent agent instances, with movement durations dropping to 30% of initial values. To our knowledge, this is the first system to apply cognitive science motor learning models to the execution layer of AI GUI agents, deployed as a ready-to-use MCP server.**

***Index Terms*—computer use agents, Model Context Protocol, muscle memory, motor learning, GUI automation, spatial caching**

I. Introduction

A human expert's 100th click on "Save" is fast and automatic. An AI computer use agent's 100th click on the same button takes exactly as long as its first. Current GUI agents have no concept of motor learning.

Recent large language models (LLMs) have enabled a new class of *computer use agents* (CUAs) that interact with desktop applications through screenshots and mouse/keyboard input [1], [2], [3]. These systems are good at deciding *what* to click based on visual understanding. But the execution layer, *how* the cursor reaches the target, is stateless. Every mouse movement is computed from scratch, even for buttons the agent has clicked hundreds of times before.

Humans work differently. Decades of cognitive science research show that repeated aimed movements follow the *Power Law of Practice* [4]: movement time decreases as a power function of repetition count. The base timing of aimed movements follows *Fitts's Law* [5], and unused motor memories fade over time following the *Ebbinghaus forgetting curve* [6].

Source code: https://github.com/santiagomd11/Agent-Forge

We present a *spatiotemporal motor memory cache* that brings this behavior to GUI agents. The system sits in the execution layer and works with any LLM backbone. It combines eight mechanisms:

1) **R-Tree spatial indexing** for proximity-based element lookup
2) **Power Law of Practice** speed-up ($T(n) = T(1) \cdot n^{-0.4}$)
3) **Fitts's Law** base movement timing
4) **WindMouse trajectory adaptation** (gravity, wind, velocity)
5) **Temporal decay**, **EMA position smoothing**, **sequence prediction**, and **cross-session persistence**

A three-layer resolution hierarchy ensures every click is captured: model-provided semantic hints (Layer 3), accessibility API auto-detection (Layer 2), and percentage-bucketed window-relative coordinates (Layer 1). Layer 1 requires only the foreground window's bounding rectangle, so *every* click builds muscle memory, even without accessibility support.

The motor memory cache is one component of a complete computer use system exposed as a Model Context Protocol (MCP) [7] server. MCP is an open standard that lets any compatible LLM client (Claude Code, Cursor, custom applications) connect to tool servers without SDK integration. Our server exposes 20 tools covering input (click, type, scroll, drag), perception (screenshots, element detection), and learned navigation (cached path traversal, action templates). The host agent calls `click(x, y, element_hint="Save")` and the cache silently accelerates repeated interactions. The system is the execution module of Agent Forge, a meta-framework for creating and running agentic workflows. This paper focuses on the motor memory innovation; the full system architecture is described in Section III.

II. Background and Related Work

A. Cognitive Science Foundations

Power Law of Practice. Newell and Rosenbloom [4] established that the time to perform a task decreases as a power function of the number of practice trials: $T(n) = T(1) \cdot n^{-\alpha}$, where α typically ranges from 0.2 to 0.6 for motor tasks. This law has been observed across diverse domains including cigar rolling, card sorting, and typing.

Fitts's Law. Fitts [5] showed that the time required for an aimed movement is logarithmically related to the ratio of distance to target width: $MT = a + b \cdot \log_2(D/W + 1)$. This remains the fundamental model of aimed movements in HCI [8], [9].

TABLE I
FEATURE COMPARISON OF MOTOR MEMORY MECHANISMS. OUR SYSTEM IS THE FIRST TO COMBINE ALL EIGHT.

Feature	Ours	Claude CUA	OpenAI CUA	UI-Mem
R-Tree spatial index	✓			
Power Law adaptation	✓			
Fitts's Law timing	✓			
WindMouse adaptation	✓			
Temporal decay	✓			
EMA position smoothing	✓			
Sequence prediction	✓			
Cross-session persistence	✓			∼

Ebbinghaus Forgetting Curve. Ebbinghaus [6] demonstrated that memory strength decays exponentially with time. We implement this as a half-life model where cache entry confidence halves every 7 days.

Motor Memory Consolidation. Willingham [10] described how procedural motor memories become automatic with repetition, moving from explicit to implicit processing. Our system models this through the WindMouse parameter adaptation: as hit count grows, cursor trajectories become straighter and faster, matching the shift from deliberate to automatic motor control.

B. AI GUI Agent Systems

Current computer use agents do not remember prior interactions at the motor level. Anthropic's Claude Computer Use [1] processes screenshots and outputs click coordinates, computing each position from scratch via LLM inference. OpenAI's Operator [2] uses a similar approach. Microsoft's UFO [3] adds multi-agent coordination; its successor UFO2 introduced session-based task memory, but neither version addresses motor-level execution adaptation. WebVoyager [11] and SeeAct [12] focus on web navigation with no motor adaptation. None of these systems learn from repeated interactions with the same target.

C. Motor Simulation and Caching Approaches

UI-Mem [13] builds a hierarchical experience memory for mobile GUI agents, storing workflows, subtask skills, and failure patterns as parameterized templates. It operates at the task planning level (what sequence of actions to take) and uses online reinforcement learning for policy improvement. It does not address motor execution (how the cursor reaches the target), spatial indexing, or movement adaptation.

Table I summarizes the feature comparison. No existing system combines even three of our eight mechanisms.

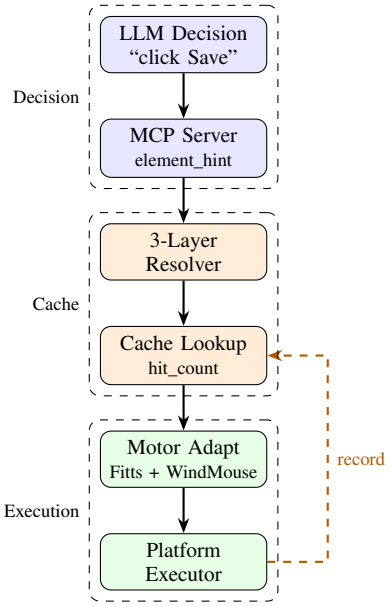

Fig. 1. Motor memory data flow within the MCP server. The cache mediates between the LLM's decision and the platform executor. A feedback loop records each interaction for future speed-up.

III. SYSTEM ARCHITECTURE

A. Overview

The system is a standalone MCP server that exposes 20 tools to any MCP-compatible LLM client. It handles perception (screenshots with automatic downscaling for vision model accuracy), input execution (click, type, scroll, drag across four OS backends), UI element detection (accessibility APIs and vision-based grounding), and learned navigation (cached path traversal, reusable action templates). Fig. 1 shows the data flow for mouse actions, where the motor memory cache sits between the LLM's decision (what to click) and the platform executor (how to reach the target). The LLM does not need to know the cache exists.

B. Three-Layer Cache Resolution

Every mouse action must resolve to a cache key: a combination of *application name* and *element hint*. We use a three-layer hierarchy so that every click is captured:

Layer 3 (Model-provided hint). When the LLM provides a semantic label such as `element_hint="Save button"`, this becomes the cache key directly. This is the highest-precision layer but depends on LLM cooperation.

Layer 2 (Accessibility API). When no hint is provided, the engine queries the operating system's accessibility tree for the UI element at the click coordinates, producing keys like `"button:Save"`. Available for native applications on Windows (UI Automation), Linux (AT-SPI2), and macOS (Accessibility API).

Layer 1 (Percentage-bucketed coordinates). The universal fallback. Requires only the foreground window's bounding rectangle. Click coordinates are converted to window-relative percentages and bucketed at 3% resolution (∼33 buckets

per axis), producing synthetic hints like `"@24%,18%"`. This layer:

- Survives **window moves** (window-relative coordinates)
- Survives **window resizes** (percentage-based)
- Works on **any application**, including games, Electron apps, and custom UIs with no accessibility support

The application name comes from the foreground window's process (e.g., `"notepad.exe"`, `"firefox"`). Remote desktop shells (RDP, VNC, terminal emulators) are excluded via a passthrough list because their foreground window is a container, not the actual application being controlled.

C. Three-Tier Cache Lookup

Given a resolved key `(app_name, hint)`, the cache performs a tiered lookup:

Tier 1 (Hot cache). An in-memory Python dictionary keyed by `"app|hint"` with $O(1)$ exact-match lookup. Sub-microsecond latency. Loaded from SQLite at startup.

Tier 2 (R-Tree spatial proximity). A SQLite R-Tree virtual table [14], [15] enables proximity search within a 50-pixel radius, scoped to the same application. Used when Tier 1 misses but coordinates overlap with a known target (e.g., a button that has moved slightly). Latency $\sim$0.1ms.

Tier 3 (Cache miss). Returns `hit_count=0`. Movement uses default WindMouse parameters.

D. Data Model

The persistent store uses SQLite with Write-Ahead Logging (WAL) for concurrent read/write safety:

- `memories`: Primary table storing `app_name`, `element_hint`, EMA-smoothed position (x, y, w, h), `hit_count`, `miss_count`, timestamps, `confidence`, and `prev_hint` for sequence tracking.
- `memories_rtree`: R-Tree spatial index on element bounding boxes.
- `sequences`: Action pair tracking (from $\rightarrow$ to) with frequency counts for workflow prediction.

An LRU eviction policy caps the cache at 10,000 entries, deleting the oldest 10% by `last_hit_ts` when capacity is reached.

E. Cross-Platform Support

The system abstracts all platform-specific operations behind a common interface. The cache algorithm itself is platform-agnostic (pure Python, stdlib `sqlite3`). Backend modules handle cursor control, window queries, and accessibility access:

- **Windows:** Win32 `SendInput` via ctypes; `GetForegroundWindow` for window info
- **WSL2:** TCP bridge daemon on Windows (port 19542) relays Win32 calls
- **Linux/X11:** `xdotool` for window queries; `/proc/{pid}/comm` for process name
- **Linux/Wayland:** Mutter RemoteDesktop DBus interface (`org.gnome.Mutter.RemoteDesktop`) for pointer and keyboard input; AT-SPI2 accessibility tree for foreground window detection; `gnome-screenshot` for screen capture
- **macOS:** AppleScript via `osascript` for window info; `cliclick` for input

Foreground window queries are TTL-cached (50ms in the engine, 100ms on subprocess-based platforms) to amortize overhead.

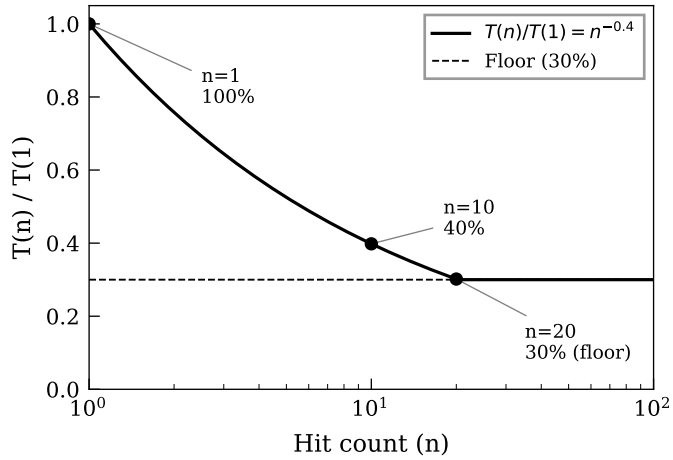

Fig. 2. Power Law of Practice curve. Movement duration decreases as $n^{-0.4}$ with a 30% floor. Key points: n=10 yields $\sim$40% of initial duration; the floor is reached at $n\approx$20. WindMouse parameter adaptation saturates at n=64.

IV. Motor Adaptation Model

This section details the six mechanisms by which repeated interactions produce faster, more efficient cursor movements.

A. Power Law of Practice

Following Newell and Rosenbloom [4], we model the speed-up of repeated aimed movements as:

$$T(n) = T_{\text{base}} \cdot n^{-\alpha}, \quad \text{floored at } T_{\text{base}} \cdot \beta \tag{1}$$

where n is the hit count (number of prior interactions with the same target), $\alpha = 0.4$ is the learning rate (mid-range for motor tasks), and $\beta = 0.3$ is the asymptotic floor (the target can never be reached faster than 30% of the initial duration).

Fig. 2 shows the adaptation curve. At n=10, movement duration has dropped to approximately 40% of initial. By $n\approx$20, it reaches the 30% floor, modeling the asymptotic performance observed in human motor skill studies.

B. Fitts's Law Base Timing

The initial movement duration T_{base} is computed using the Shannon formulation of Fitts's Law [5], [8]:

$$MT = a + b \cdot \log_2\left(\frac{D}{W} + 1\right) \tag{2}$$

with $a = 0.05\,\text{s}$ (base reaction time), $b = 0.11\,\text{s}$ (movement time coefficient), and a floor of 0.07 s. Gaussian jitter ($\sigma_a = 0.01$, $\sigma_b = 0.015$) is added to both coefficients to produce human-like variability. The Power Law factor from Eq. 1 is then applied to the resulting duration.

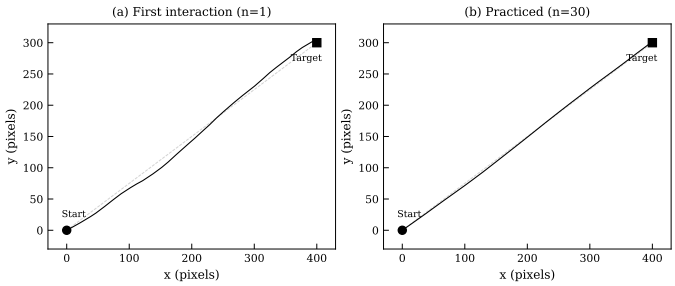

Fig. 3. WindMouse cursor trajectories. (a) First interaction (n=1): wobbly, exploratory path. (b) After 30 repetitions: nearly straight, confident path. Both share the same start and end points.

C. WindMouse Trajectory Adaptation

Cursor trajectories are generated using the WindMouse model, which simulates a particle under two forces: *gravity* (directed toward the target) and *wind* (random perturbation). The model operates in two phases:

- **Ballistic phase** ($d \geq d_h$): Active wind perturbation produces curved, exploratory paths.
- **Homing phase** ($d < d_h$): Wind decays exponentially; gravity dominates, producing precise final approach.

where d is the remaining distance and $d_h = 12\,\text{px}$ is the homing threshold.

As practice count increases, WindMouse parameters adapt via logarithmic scaling (saturating at $\sim$64 hits):

$$f = \min\left(1.0,\ \frac{\log_2(n)}{6.0}\right) \tag{3}$$

$$G(n) = 9.0 + 11.0 \cdot f \qquad \text{(gravity: } 9 \rightarrow 20\text{)} \tag{4}$$

$$W(n) = 3.0 \cdot (1 - 0.7 \cdot f) \qquad \text{(wind: } 3 \rightarrow 0.9\text{)} \tag{5}$$

$$V(n) = 20.0 + 15.0 \cdot f \qquad \text{(max vel: } 20 \rightarrow 35\text{)} \tag{6}$$

In practice, this means that repeated trajectories become straighter (higher gravity), less wobbly (lower wind), and faster (higher maximum velocity). This matches the transition from deliberate to automatic motor control observed in humans [10]. Fig. 3 illustrates the difference.

D. Temporal Decay

Cache entries lose confidence over time, modeling the Ebbinghaus forgetting curve [6]:

$$c(t) = c_0 \cdot 2^{-t/\tau} \tag{7}$$

where $\tau = 604{,}800\,\text{s}$ (7 days) is the half-life. Entries with $c(t) < 0.1$ are treated as forgotten (cache miss). Each successful interaction resets $c_0 = 1.0$. Additionally, each *miss* (target not found where expected) penalizes confidence by -0.3, providing rapid adaptation when UI layouts change.

E. EMA Position Smoothing

UI elements may drift between sessions due to window resizing, layout changes, or dynamic content. We track element positions using an Exponential Moving Average:

$$\hat{x}_{n+1} = \hat{x}_n + \alpha \cdot (x_{\text{obs}} - \hat{x}_n) \tag{8}$$

with $\alpha = 0.3$. This smoothing absorbs drift while preserving the learned position of stable elements. The same EMA is applied to element width and height.

F. Sequence Prediction

The system tracks action transitions as ordered pairs: when the user clicks target B after target A, the pair $(A \rightarrow B)$ is recorded with a frequency count. The `predict_next(app, current_hint)` function returns the most frequent successor, enabling anticipatory pre-computation of the next target's cache entry. Example from live usage: after the sequence "WhatsApp taskbar icon $\rightarrow$ Andrés P Scale chat" occurs twice, the system predicts this transition with high confidence.

V. Implementation

The system is implemented in Python with zero external dependencies for the core cache (`sqlite3` is part of the standard library). Key implementation details:

Storage. SQLite with Write-Ahead Logging (WAL) mode and `NORMAL` synchronous setting balances durability and performance. The R-Tree virtual table is compiled into SQLite by default on all major platforms.

MCP server. The entire system is packaged as an MCP [7] server exposing 20 tools over stdio or SSE transport. An LLM client connects by pointing its MCP configuration at the server binary; no SDK integration or code changes are required in the host agent. Screenshots are automatically downscaled to a resolution that balances vision model accuracy with coordinate precision (e.g., 1366 px wide for a 3840 px display). Mouse action tools accept an optional `element_hint` parameter that feeds the motor memory cache; the engine transparently performs cache resolution, lookup, and recording around each action.

Navigation acceleration. Two higher-level mechanisms skip LLM roundtrips entirely. *Navigate-to* performs BFS path-finding through the sequence graph and executes via cached positions ($\sim$55 ms). *Action templates* are named multi-step workflows (click, type, key-press, wait) replayed from the database; a 5-step template completes in under 1 s vs. $\sim$10$-$20 s with LLM processing.

Resolution resilience. Cache entries track both window dimensions and screen resolution at recording time. When the screen resolution changes (e.g., switching monitors or DPI settings), cached coordinates are proportionally rescaled. When only the window is resized, entries are used as-is if coordinates remain in bounds, or treated as misses otherwise. This avoids the distortion that proportional rescaling causes on fixed-grid UIs like spreadsheets.

Scale. The implementation comprises approximately 1,100 lines for the cache module (including navigation, resolution resilience, and action templates), 820 lines for the engine facade, and 500 lines for MCP server integration. A test suite of 352 unit tests covers cache CRUD, spatial proximity, temporal decay, power law adaptation, sequence tracking,

TABLE II
CACHE GROWTH ACROSS THREE SESSIONS. SESSION 2 WAS PERFORMED BY AN INDEPENDENT AGENT WITH ZERO SHARED CONTEXT.

Metric	Session 1	Session 2	Session 3
Total entries	7	7	12
Cache hits	0	3	5
Avg hit_count	1.0	1.4	1.5
Unique apps	3	3	3
Sequences recorded	5	7	9
New targets	7	0	5

persistence, eviction, three-layer resolution, passthrough exclusion, percentage-bucket alignment, navigation path-finding, resolution rescaling, DPI-aware accessibility, and action template workflows.

VI. EVALUATION

A. Experimental Setup

We evaluate the system on two platforms:

Windows 11 + WSL2. 3840×2400 display (DPI 288), engine on WSL2 with a native Windows bridge daemon for input execution. Test applications include WhatsApp Desktop, with a multi-session workflow: open application from taskbar, navigate to contact, send message, close application, reopen, and edit the sent message.

Ubuntu Linux (GNOME/Wayland). 1366×853 display, engine running natively with Mutter RemoteDesktop for input and AT-SPI2 for window detection. 40 end-to-end tests across six applications (Chrome, GNOME Text Editor, Terminal, Nautilus file manager, GNOME Settings, and desktop shell), covering click, double-click, right-click, drag, type, scroll, and keyboard shortcuts.

B. Cache Growth Across Sessions

Table II shows cache metrics across three evaluation sessions. Session 1 is a cold start (all misses). Session 2, performed by an *independent agent instance* with no shared context, achieves cache hits on all previously-seen targets. Session 3 introduces a different contact, demonstrating that independent targets accumulate separate hit counts while shared elements (taskbar icon, message input) benefit from combined usage.

On the Linux platform, the 40 end-to-end tests produced 28 unique cache entries across 6 applications (Chrome: 11, Nautilus: 5, GNOME Terminal: 4, GNOME Text Editor: 4, GNOME Settings: 2, GNOME Shell: 2). All entries were correctly keyed by real application name via AT-SPI2 foreground window detection. Elements with 3+ hits (e.g., browser tabs, address bar, back button) were successfully accessed via cached navigation, executing multi-step click sequences without any LLM roundtrips.

C. Movement Duration Adaptation

Fig. 4 shows the theoretical movement duration for a representative target (500 px distance, 40 px width) as a function of interaction count. The Power Law reduction from Eq. 1 produces rapid initial improvement that asymptotes at the 30% floor after approximately 20 interactions.

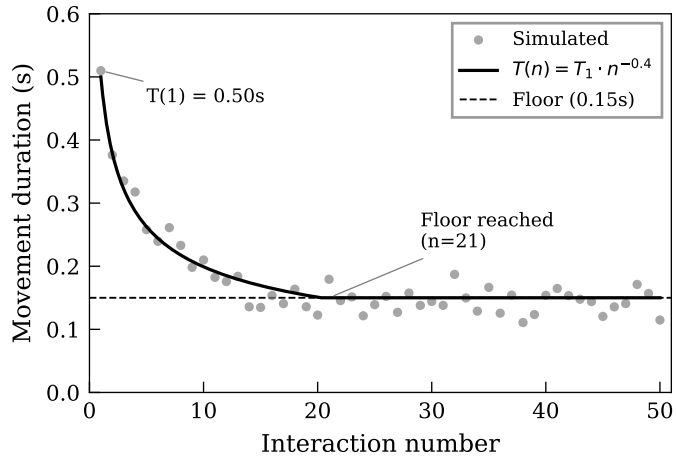

Fig. 4. Simulated movement duration vs. interaction count for a representative target (500 px distance, 40 px width). Scatter points are sampled from the Power Law model with Gaussian noise (σ=0.02 s). Duration floors at 30% of initial value.

D. Qualitative Observations

Cross-agent persistence. Agent B benefits from Agent A's prior interactions without any explicit knowledge transfer. The shared SQLite database carries the learned positions forward automatically.

Resize resilience. Percentage-based coordinate bucketing (Layer 1) correctly resolves cache hits after window resizing. A target at 25% of an 800 px window (200 px) maps to the same bucket as 25% of a 1200 px window (300 px).

Passthrough exclusion. Remote desktop shells (RDP, VNC) are correctly excluded from caching, so cache entries are not associated with a container application instead of the actual UI being controlled.

Wayland compatibility. On GNOME Wayland, standard tools like `xdotool` cannot detect the foreground window or inject input. The system uses AT-SPI2 (accessibility tree traversal) for window detection and the Mutter RemoteDesktop DBus interface for absolute pointer positioning. Cache entries are correctly keyed by application name across both X11 and Wayland sessions.

E. Limitations

The current evaluation has several limitations. First, we have not conducted a formal controlled study comparing movement time and trajectory quality with and without the motor memory cache. Second, the Power Law exponent ($\alpha = 0.4$) and floor ($\beta = 0.3$) are fixed values from the literature rather than fitted to observed agent behavior. Third, the evaluation is single-user on a limited set of applications. Fourth, cursor trajectory "naturalness" is inherently subjective without a formal human-likeness metric.

VII. Discussion

Works with any LLM. The cache operates in the execution layer and works with Claude, GPT-4, open-source models, or any other decision framework. It does not change *what* the agent clicks, only how fast the cursor gets there.

Trajectories converge naturally. As hit count grows, the WindMouse parameter adaptation produces trajectories that approach straight lines with smooth velocity profiles. This resembles the minimum-jerk trajectory seen in expert human pointing. We did not program this directly; it emerges from the parameter adaptation.

Shared memory across agents. Because the cache is SQLite on disk, multiple independent agent instances share motor memory automatically. Agent B moves faster to targets that Agent A has already used.

Living system. All parameters ($\alpha = 0.4$, $\beta = 0.3$, $\tau =$ 7 days, EMA $\alpha = 0.3$, radius = 50 px) are configurable and can be made adaptive as more usage data is collected.

Part of a broader framework. The computer use system is the "eyes and hands" module of Agent Forge, a meta-framework for generating agentic workflows through a 7-step orchestrator. Agent Forge alone generates text and code workflows; combined with the computer use server, it targets fully autonomous desktop automation where generated workflows can be executed without human intervention. This integration is ongoing work.

VIII. Conclusion and Future Work

We have presented a cognitive motor learning system for autonomous desktop agents, deployed as a ready-to-use MCP server. Its key contribution is a spatiotemporal motor memory cache grounded in cognitive science (Power Law of Practice, Fitts's Law, Ebbinghaus temporal decay). The three-layer resolution hierarchy and eight adaptation mechanisms let any MCP-compatible agent develop muscle memory that persists across sessions and transfers between agent instances.

Future work includes: (1) formal A/B studies comparing agent performance with and without the cache; (2) adaptive α fitting from observed data per element type; (3) anticipatory cursor movement toward predicted targets before the LLM completes its decision; (4) a shared cloud cache for fleet-wide motor memory; (5) integration between the computer use server and Agent Forge's workflow orchestrator for fully autonomous desktop task execution; and (6) richer accessibility tree information for improved Layer 2 resolution.

References

[1] Anthropic, "Developing a computer use model," Anthropic Blog, 2024. [Online]. Available: https://www.anthropic.com/news/developing-computer-use

[2] OpenAI, "Introducing operator," OpenAI Blog, 2025. [Online]. Available: https://openai.com/index/introducing-operator/

[3] C. Zhang, L. Li, S. He, X. Zhang, B. Qiao, S. Qin, M. Ma, Y. Kang, Q. Lin, S. Rajmohan, D. Zhang, and Q. Zhang, "UFO: A UI-focused agent for Windows OS interaction," *arXiv preprint arXiv:2402.07939*, 2024.

[4] A. Newell and P. S. Rosenbloom, "Mechanisms of skill acquisition and the law of practice," in *Cognitive Skills and Their Acquisition*, J. R. Anderson, Ed. Lawrence Erlbaum Associates, 1981, pp. 1–55.

[5] P. M. Fitts, "The information capacity of the human motor system in controlling the amplitude of movement," *Journal of Experimental Psychology*, vol. 47, no. 6, pp. 381–391, 1954.

[6] H. Ebbinghaus, *Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie*. Leipzig: Duncker & Humblot, 1885, english translation: *Memory: A Contribution to Experimental Psychology*, 1913.

[7] Anthropic, "Model context protocol," Anthropic, 2024. [Online]. Available: https://modelcontextprotocol.io/

[8] I. S. MacKenzie, "Fitts' law as a research and design tool in human-computer interaction," *Human-Computer Interaction*, vol. 7, pp. 91–139, 1992.

[9] R. W. Soukoreff and I. S. MacKenzie, "Towards a standard for pointing device evaluation, perspectives on 27 years of Fitts' law research in HCI," *International Journal of Human-Computer Studies*, vol. 61, no. 6, pp. 751–789, 2004.

[10] D. B. Willingham, "A neuropsychological theory of motor skill learning," *Psychological Review*, vol. 105, no. 3, pp. 558–584, 1998.

[11] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, "WebVoyager: Building an end-to-end web agent with large multimodal models," *arXiv preprint arXiv:2401.13919*, 2024.

[12] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "SeeAct: GPT-4V(ision) is a generalist web agent, if grounded," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[13] H. Xiao, G. Wang, H. Wang, S. Liu, Y. Chai, Y. Pan, Y. Zhou, X. Chen, Y. Wen, and H. Li, "UI-Mem: Self-evolving experience memory for online reinforcement learning in mobile GUI agents," *arXiv preprint arXiv:2602.05832*, 2026.

[14] A. Guttman, "R-Trees: A dynamic index structure for spatial searching," in *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*, 1984, pp. 47–57.

[15] SQLite, "The SQLite R*Tree module," sqlite.org. [Online]. Available: https://www.sqlite.org/rtree.html